FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Basister, Kian Shane Q. Calingasan, Dwayne Giusseppe P.
Section:	BSCSSE – AN41
Professor:	Ms. Crisola Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- **What each partner contributed**
  - We focused on defining the student profiles, identifying learning errors, and organizing the case base. My partner helped implement the similarity calculation and case-saving process, while we worked together on adapting feedback and testing the program.
- **What challenges you encountered**
  - One challenge was deciding which student characteristics should carry the most weight when comparing cases. We addressed this by prioritizing error patterns and checking whether the retrieved case provided suitable guidance for the new student.
- **What you learned about CBR**
  - We learned that case-based reasoning solves new problems by retrieving similar experiences, adapting their solutions, reviewing the results, and retaining new cases. We also learned that saving a recommended intervention does not automatically mean it was successful, so recording its outcome matters.

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.

Student submits a math solution with repeated errors.

System retrieves past students with similar error patterns (e.g., misusing formulas).

It adapts feedback and personalized lessons based on successful interventions.

Learning:

System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

```
PS C:\Users\Kian\Desktop\CODES\IS\TECHNICALS\TECHNICAL3> py main.py
TA3 | INTELLIGENT TUTORING SYSTEM | CASE-BASED REASONING
Educational demonstration; all initial cases are fictional.
Loaded cases: 5

1. NEW PROBLEM
A Grade 8 student repeatedly adds instead of subtracting a constant.
Topic: linear_equations | Grade: 8 | Error frequency: 3/5
Observed errors: inverse_operation
  3x + 6 = 18 -> 3x = 24 -> x = 8 (incorrect)
  2x + 4 = 12 -> 2x = 16 -> x = 8 (incorrect)
  5x + 5 = 20 -> 5x = 25 -> x = 5 (incorrect)

2. RETRIEVE | SIMILARITY RANKING
C01:  95.00% | linear_equations | inverse_operation
C02:  55.00% | linear_equations | sign_error
C03:  53.00% | linear_equations | distribution, sign_error
C05:  23.00% | geometry | formula_misuse
C04:  22.00% | fractions | unlike_denominators
Retrieved: C01 (95.00%)
Past solution: Model the equation as a balance, then use guided practice.
Past practice count: 3

3. REUSE AND ADAPT
Model the equation as a balance, then use guided practice.
Feedback: Use the opposite operation on BOTH sides of the equation.
Use Grade 8 examples for linear equations.
Work through two guided examples before independent practice.
Example: 3x + 6 = 18 -> 3x = 12 -> x = 4. Check: 3(4) + 6 = 18.

4. REVISE | FOLLOW-UP CHECK
Simulated follow-up result: 4/5 correct (not a real student outcome).
Outcome: successful
FINAL SOLUTION: 4 practice problems. Work through two guided examples before independent practice.

5. RETAIN | UPDATED CASE BASE
New case saved: C06
C01 | linear_equations | inverse_operation | successful
C02 | linear_equations | sign_error | successful
C03 | linear_equations | distribution, sign_error | successful
C04 | fractions | unlike_denominators | successful
C05 | geometry | formula_misuse | successful
C06 | linear_equations | inverse_operation | successful
Total cases: 6 | Saved in: case_base.json
PS C:\Users\Kian\Desktop\CODES\IS\TECHNICALS\TECHNICAL3>
```

```
import argparse
from copy import deepcopy
import json
from pathlib import Path
import tempfile

DATA_FILE = Path(__file__).with_name("case_base.json")
WEIGHTS = {"topic": 0.30, "errors": 0.45, "grade": 0.15, "frequency": 0.10}
LESSONS = {
    "inverse_operation": "Use the opposite operation on BOTH sides of the equation.",
    "sign_error": "Write the sign of every term before combining or moving terms.",
    "distribution": "Multiply the outside factor by EVERY term inside parentheses.",
    "unlike_denominators": "Find a common denominator before adding fractions.",
    "formula_misuse": "Identify the shape and label each value before choosing a formula.",
}
TOPIC_ERRORS = {
    "linear_equations": ["inverse_operation", "sign_error", "distribution"],
    "fractions": ["unlike_denominators"],
```

```
    "geometry": ["formula_misuse"],
}

def make_case(case_id,  topic, errors, grade, frequency, strategy,  practice):
    return {
        "id": case_id,
        "problem": {"topic": topic, "errors": errors, "grade": grade,
                    "frequency": frequency},
        "solution": {"strategy": strategy,  "practice_count":  practice,
                     "feedback": [LESSONS[error] for error in errors]},
        "outcome": {"status": "successful", "correct": 4, "total": 5,
                    "source": "fictional seed example"},
    }

SEED_CASES = [
    make_case("C01", "linear_equations", ["inverse_operation"],  7, 2,
              "Model the equation as a balance, then use guided practice.", 3),
    make_case("C02", "linear_equations", ["sign_error"], 8, 3,
              "Highlight signed terms and check each intermediate step.", 4),
    make_case("C03", "linear_equations", ["distribution", "sign_error"], 8, 2,
              "Expand brackets one term at a time before solving.", 4),
    make_case("C04", "fractions", ["unlike_denominators"],  7, 3,
              "Use fraction strips to demonstrate equal-sized parts.", 4),
    make_case("C05", "geometry", ["formula_misuse"], 8, 2,
              "Label a diagram and explain the selected area formula.", 3),
]

NEW_PROBLEM = {
    "topic": "linear_equations", "errors": ["inverse_operation"],
    "grade": 8, "frequency": 3,
    "description": "A Grade 8 student repeatedly adds instead of subtracting a constant.",
    "submitted_work": ["3x + 6 = 18 -> 3x = 24 -> x = 8 (incorrect)",
                       "2x + 4 = 12 -> 2x = 16 -> x = 8 (incorrect)",
                       "5x + 5 = 20 -> 5x = 25 -> x = 5 (incorrect)"],
}

def validate_problem(problem):
    """Validate the teacher-labelled profile; this demo does not parse algebra."""
    topics = {case["problem"]["topic"] for case in SEED_CASES}
    if problem.get("topic") not in topics:
        raise ValueError("Unsupported topic.")
    errors = problem.get("errors")
    if not isinstance(errors, list) or not errors or any(e not in LESSONS for e in errors):
        raise ValueError("Select at least one supported error.")
```

```
    if any(error not in TOPIC_ERRORS[problem["topic"]] for error in errors):
        raise ValueError("The selected error does not belong to this topic.")
    for field, low, high in (("grade", 1, 12), ("frequency", 1, 5)):
        value = problem.get(field)
        if type(value) is not int or not low <= value <= high:
            raise ValueError(f"{field} must be an integer from {low} to {high}.")

def similarity(new, old):
    """Weighted score in [0, 1]; error overlap uses Jaccard similarity."""
    first, second = set(new["errors"]), set(old["errors"])
    components = {
        "topic": float(new["topic"] == old["topic"]),
        "errors": len(first & second) / len(first | second),
        "grade": max(0.0, 1 - abs(new["grade"] - old["grade"]) / 5),
        "frequency": max(0.0, 1 - abs(new["frequency"] - old["frequency"]) / 5),
    }
    return sum(WEIGHTS[key] * value for key, value in components.items())

def retrieve(problem, cases):
    # Pending interventions remain in the case base, but are not success evidence.
    candidates = [case for case in cases if case["outcome"]["status"] == "successful"]
    if not candidates:
        raise ValueError("No successful past cases are available for retrieval.")
    return sorted(((similarity(problem, case["problem"]), case) for case in candidates),
                  key=lambda item: (-item[0], item[1]["id"]))

def adapt(problem, retrieved):
    """Reuse the old plan, then revise its feedback and workload for this learner."""
    plan = deepcopy(retrieved["solution"])
    old = retrieved["problem"]
    if problem["topic"] != old["topic"] or not set(problem["errors"]) & set(old["errors"]):
        plan["strategy"] = "Explain the target skill, model one example, then guide practice."
    plan["feedback"] = [LESSONS[error] for error in dict.fromkeys(problem["errors"])]
    extra = max(0, problem["frequency"] - old["frequency"])
    plan["practice_count"] = min(10, plan["practice_count"] + extra)
    plan["support"] = ("Work through two guided examples before independent practice."
                       if problem["frequency"] >= 3 else
                       "Work through one guided example before independent practice.")
    plan["level"] = f"Use Grade {problem['grade']} examples for {problem['topic'].replace('_', ' ')}."
    if problem["topic"] == "linear_equations" and "inverse_operation" in problem["errors"]:
        plan["worked_example"] = "3x + 6 = 18 -> 3x = 12 -> x = 4. Check: 3(4) + 6 = 18."
    else:
        plan.pop("worked_example", None)
    return plan
```

```
def revise(plan, correct, total=5, simulated=False):
    """Review a follow-up quiz; a weak result triggers additional support."""
    if correct is None:
        return {"status": "pending", "correct": None, "total": total,
                "source": "follow-up not yet recorded"}
    if type(correct) is not int or not 0 <= correct <= total:
        raise ValueError("The follow-up score must be between zero and the quiz total.")
    passed = correct / total >= 0.8
    if not passed:
        plan["practice_count"] = min(10, plan["practice_count"] + 2)
        plan["support"] = "Reteach with concrete examples and teacher guidance, then reassess."
    return {"status": "successful" if passed else "needs_review",
            "correct": correct, "total": total,
            "source": "simulated demo quiz" if simulated else "user-entered follow-up quiz"}

def load_cases(path):
    if not path.exists():
        return deepcopy(SEED_CASES)
    cases = json.loads(path.read_text(encoding="utf-8"))
    if not isinstance(cases, list) or not cases:
        raise ValueError("Case base must be a nonempty JSON list.")
    ids = set()
    for case in cases:
        validate_problem(case["problem"])
        if case["id"] in ids:
            raise ValueError("Duplicate case ID in saved data.")
        ids.add(case["id"])
        if not isinstance(case["solution"]["strategy"], str):
            raise ValueError("Invalid saved strategy.")
        count = case["solution"]["practice_count"]
        if type(count) is not int or not 1 <= count <= 10:
            raise ValueError("Invalid saved practice count.")
        if case["outcome"]["status"] not in {"successful", "pending", "needs_review"}:
            raise ValueError("Invalid saved outcome status.")
    return cases

def retain(cases, problem, plan, outcome, retrieved_id, path):
    # Avoid saving the identical demonstration again on every run.
    for case in cases:
        if case["problem"] == problem and case["outcome"] == outcome:
            return case, False
    number = 1
    while f"C{number:02d}" in {case["id"] for case in cases}:
```

```
        number += 1
    new_case = {"id": f"C{number:02d}", "problem": deepcopy(problem),
                "solution": deepcopy(plan), "outcome": deepcopy(outcome),
                "retrieved_from": retrieved_id}
    updated = cases + [new_case]
    # Replace atomically so an interrupted write does not leave partial JSON.
    temp_path = None
    try:
        with tempfile.NamedTemporaryFile(mode="w", encoding="utf-8", dir=path.parent,
                                         suffix=".tmp", delete=False) as handle:
            temp_path = Path(handle.name)
            json.dump(updated, handle, indent=2)
            handle.write("\n")
        temp_path.replace(path)
    finally:
        if temp_path is not None:
            temp_path.unlink(missing_ok=True)
    cases.append(new_case)
    return new_case, True

def ask_number(prompt, low, high):
    while True:
        try:
            value = int(input(prompt))
            if low <= value <= high:
                return value
        except ValueError:
            pass
        print(f"Please enter a whole number from {low} to {high}.")

def interactive_problem():
    topics = ["linear_equations", "fractions", "geometry"]
    for index, topic in enumerate(topics, 1):
        print(f"{index}. {topic.replace('_', ' ')}")
    topic = topics[ask_number("Choose topic: ", 1, len(topics)) - 1]
    errors = TOPIC_ERRORS[topic]
    for index, error in enumerate(errors, 1):
        print(f"{index}. {error.replace('_', ' ')}")
    error = errors[ask_number("Choose the main observed error: ", 1, len(errors)) - 1]
    return {"topic": topic, "errors": [error],
            "grade": ask_number("Grade (1-12): ", 1, 12),
            "frequency": ask_number("Occurrences in the last 5 attempts (1-5): ", 1, 5),
            "description": input("Describe the student's difficulty: ").strip() or
                           "Student needs support with the selected skill."}
```

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--interactive",  action="store_true",  help="enter a student profile")
    parser.add_argument("--data",  type=Path, default=DATA_FILE, help="case base JSON path")
    args = parser.parse_args()
    try:
        cases = load_cases(args.data)
        print("TA3 | INTELLIGENT TUTORING SYSTEM | CASE-BASED REASONING")
        print("Educational demonstration; all initial cases are fictional.")
        print(f"Loaded cases: {len(cases)}")
        problem = interactive_problem() if args.interactive else deepcopy(NEW_PROBLEM)
        validate_problem(problem)
        print("\n1. NEW PROBLEM")
        print(problem["description"])
        print(f"Topic: {problem['topic']} | Grade: {problem['grade']} | "
              f"Error frequency: {problem['frequency']}/5")
        print("Observed errors: " + ", ".join(problem["errors"]))
        for step in problem.get("submitted_work", []):
            print("  " + step)

        ranking = retrieve(problem, cases)
        print("\n2. RETRIEVE | SIMILARITY RANKING")
        for score, case in ranking:
            print(f"{case['id']}: {score * 100:6.2f}% | {case['problem']['topic']} | "
                  + ", ".join(case["problem"]["errors"]))
        best_score, best_case = ranking[0]
        print(f"Retrieved: {best_case['id']} ({best_score:.2%})")
        print("Past solution: " + best_case["solution"]["strategy"])
        print(f"Past practice count: {best_case['solution']['practice_count']}")
        print("\n3. REUSE AND ADAPT")
        plan = adapt(problem, best_case)
        print(plan["strategy"])
        for feedback in plan["feedback"]:
            print("Feedback: " + feedback)
        print(plan["level"])
        print(plan["support"])
        if "worked_example" in plan:
            print("Example: " + plan["worked_example"])

        print("\n4. REVISE | FOLLOW-UP CHECK")
        if args.interactive:
            print("Enter a follow-up quiz score, or -1 if it has not been taken.")
            score = ask_number("Correct answers out of 5 (-1 to 5): ", -1, 5)
```

```
            score = None if score == -1 else score
        else:
            score = 4
            print("Simulated follow-up result: 4/5 correct (not a real student outcome).")
        outcome = revise(plan, score, simulated=not args.interactive)
        print(f"Outcome: {outcome['status']}")
        print(f"FINAL SOLUTION: {plan['practice_count']} practice problems. {plan['support']}")

        saved, added = retain(cases, problem, plan, outcome, best_case["id"], args.data)
        print("\n5. RETAIN | UPDATED CASE BASE")
        print(f"New case saved: {saved['id']}" if added else
              f"Identical case already saved: {saved['id']} (no duplicate added)")
        for case in cases:
            print(f"{case['id']} | {case['problem']['topic']} | "
                  f"{', '.join(case['problem']['errors'])} | {case['outcome']['status']}")
        print(f"Total cases: {len(cases)} | Saved in: {args.data.name}")
    except (OSError, ValueError, KeyError, TypeError) as error:
        parser.exit(1, f"Error: {error}\nCheck the inputs and saved JSON; existing data was not
reset.\n")
    except (EOFError, KeyboardInterrupt):
        parser.exit(1, "\nInput cancelled; no new case saved.\n")

if __name__ == "__main__":
    main()
```